

estack printable cheatsheet

One page map of what you can invoke, what euge-mode routes, and the principles behind the stack.

1. Start here Use euge-mode for serious engineering work. It is the router.	2. Direct skills Invoke a named workflow when you already know the shape you want.	3. Helpers how, why, TypeScript, and unslop are narrow enough for model matching.	4. Principles Treat principle-* files as routed references, not normal commands.
Invokable skills commands and helpers	euge-mode playbooks router paths	Principles routed references	
euge-mode Default router for non-trivial work. Picks playbooks, principles, and helpers.	investigation Read-only question: how, why, are we sure, should we do X.	Laziness Protocol Smallest change that solves the problem.	
how Explains subsystem structure, runtime flow, ownership, and layering.	bug fix Reproduce, root-cause, fix, and verify a defect.	Foundational Thinking Get data shape and shared ownership right first.	
why Investigates history, rationale, tradeoffs, regressions, and thresholds.	perf issue Trace measured slowness and improve against a baseline.	Redesign From First Principles Integrate requirements as if known from day one.	
architect Sketches types, signatures, and module boundaries before implementation.	runtime forensics Diagnose a live runtime symptom from instrumentation.	Subtract Before You Add Remove dead weight before building more.	
arena Compares parallel candidates for one task when several approaches could work.	trace forensics Diagnose a captured profile, trace, spindump, or heap snapshot.	Minimize Reader Load Collapse needless layers and hidden state.	
interrogate Adversarial multi-model review for a diff, design, or blind spots.	feature Build new or changed behavior from a named data shape.	Outcome-Oriented Execution Optimize for the verified end state.	
figure-it-out Before hard ambiguous work: designs a custom auditable playbook.	refactoring Preserve behavior while changing structure.	Experience First Choose user experience over convenience.	
reflect After a task: captures durable lessons and routes skill edits.	prototype Build a cheap sketch to decide direction.	Exhaust the Design Space Compare concrete options before hard choices.	
automate-me Turns your working style into a personal mode skill.	visual parity Match UI pixels or styling between versions.	Build the Lever Script or generate repetitive work.	
show-me-your-work Keeps a TSV decision trail for long or review-later work.	authoring a skill Write or edit a SKILL.md.	Boundary Discipline Validate at boundaries; trust typed internals.	
tdd Writes a failing regression test first when the test path is cheap.	eval Test how a skill or prompt change affects behavior.	Type System Discipline Make illegal states unrepresentable.	
typescript-best-practices TypeScript rules grounded in type-system discipline.	autonomous run Drive a long task without stopping.	Make Operations Idempotent Reruns and retries converge safely.	
unslop Cleans writing and removes AI tells.	session pickup Resume prior work from transcript, branch, or URL.	Migrate Callers Then Delete Legacy APIs Move callers and remove old APIs together.	
	multi-phase plan Plan work across phases or stacked PRs.	Separate Before Serializing Shared State Eliminate shared writes before locks.	
	opening a PR Publication workflow after other playbooks.	Prove It Works Verify the real artifact, not a proxy.	
		Fix Root Causes Reproduce, trace why, and fix the cause.	
		Guard the Context Window Delegate bulk; keep summaries tight.	
		Never Block on the Human Proceed on reversible work.	
		Encode Lessons in Structure Turn repeated instructions into checks or tools.	